every edge $(u, v) \in E$, there is a shortest-paths tree rooted at s that contains (u, v) and another shortest-paths tree rooted at s that does not contain (u, v).

22.5-3
Modify the proof of Lemma 22.10 to handle cases in which shortest-path weights are ∞ or $-\infty$.

22.5-4
Let $G = (V, E)$ be a weighted, directed graph with source vertex s, and let G be initialized by INITIALIZE-SINGLE-SOURCE(G, s). Prove that if a sequence of relaxation steps sets $s.\pi$ to a non-NIL value, then G contains a negative-weight cycle.

22.5-5
Let $G = (V, E)$ be a weighted, directed graph with no negative-weight edges. Let $s \in V$ be the source vertex, and suppose that $v.\pi$ is allowed to be the predecessor of v on *any* shortest path to v from source s if $v \in V - \{s\}$ is reachable from s, and NIL otherwise. Give an example of such a graph G and an assignment of π values that produces a cycle in G_π. (By Lemma 22.16, such an assignment cannot be produced by a sequence of relaxation steps.)

22.5-6
Let $G = (V, E)$ be a weighted, directed graph with weight function $w : E \to \mathbb{R}$ and no negative-weight cycles. Let $s \in V$ be the source vertex, and let G be initialized by INITIALIZE-SINGLE-SOURCE(G, s). Use induction to prove that for every vertex $v \in V_\pi$, there exists a path from s to v in G_π and that this property is maintained as an invariant over any sequence of relaxations.

22.5-7
Let $G = (V, E)$ be a weighted, directed graph that contains no negative-weight cycles. Let $s \in V$ be the source vertex, and let G be initialized by INITIALIZE-SINGLE-SOURCE(G, s). Prove that there exists a sequence of $|V| - 1$ relaxation steps that produces $v.d = \delta(s, v)$ for all $v \in V$.

22.5-8
Let G be an arbitrary weighted, directed graph with a negative-weight cycle reachable from the source vertex s. Show how to construct an infinite sequence of relaxations of the edges of G such that every relaxation causes a shortest-path estimate to change.

Problems

22-1 Yen's improvement to Bellman-Ford

The Bellman-Ford algorithm does not specify the order in which to relax edges in each pass. Consider the following method for deciding upon the order. Before the first pass, assign an arbitrary linear order $v_1, v_2, \ldots, v_{|V|}$ to the vertices of the input graph $G = (V, E)$. Then partition the edge set E into $E_f \cup E_b$, where $E_f = \{(v_i, v_j) \in E : i < j\}$ and $E_b = \{(v_i, v_j) \in E : i > j\}$. (Assume that G contains no self-loops, so that every edge belongs to either E_f or E_b.) Define $G_f = (V, E_f)$ and $G_b = (V, E_b)$.

a. Prove that G_f is acyclic with topological sort $\langle v_1, v_2, \ldots, v_{|V|} \rangle$ and that G_b is acyclic with topological sort $\langle v_{|V|}, v_{|V|-1}, \ldots, v_1 \rangle$.

Suppose that each pass of the Bellman-Ford algorithm relaxes edges in the following way. First, visit each vertex in the order $v_1, v_2, \ldots, v_{|V|}$, relaxing edges of E_f that leave the vertex. Then visit each vertex in the order $v_{|V|}, v_{|V|-1}, \ldots, v_1$, relaxing edges of E_b that leave the vertex.

b. Prove that with this scheme, if G contains no negative-weight cycles that are reachable from the source vertex s, then after only $\lceil |V| / 2 \rceil$ passes over the edges, $v.d = \delta(s, v)$ for all vertices $v \in V$.

c. Does this scheme improve the asymptotic running time of the Bellman-Ford algorithm?

22-2 Nesting boxes

A d-dimensional box with dimensions $(x_1, x_2, \ldots, x_d)$ ***nests*** within another box with dimensions $(y_1, y_2, \ldots, y_d)$ if there exists a permutation π on $\{1, 2, \ldots, d\}$ such that $x_{\pi(1)} < y_1, x_{\pi(2)} < y_2, \ldots, x_{\pi(d)} < y_d$.

a. Argue that the nesting relation is transitive.

b. Describe an efficient method to determine whether one d-dimensional box nests inside another.

c. You are given a set of n d-dimensional boxes $\{B_1, B_2, \ldots, B_n\}$. Give an efficient algorithm to find the longest sequence $\langle B_{i_1}, B_{i_2}, \ldots, B_{i_k} \rangle$ of boxes such that B_{i_j} nests within $B_{i_{j+1}}$ for $j = 1, 2, \ldots, k - 1$. Express the running time of your algorithm in terms of n and d.

22-3 Arbitrage

Arbitrage is the use of discrepancies in currency exchange rates to transform one unit of a currency into more than one unit of the same currency. For example, suppose that one U.S. dollar buys 64 Indian rupees, one Indian rupee buys 1.8 Japanese yen, and one Japanese yen buys 0.009 U.S. dollars. Then, by converting currencies, a trader can start with 1 U.S. dollar and buy $64 \times 1.8 \times 0.009 = 1.0368$ U.S. dollars, thus turning a profit of 3.68%.

Suppose that you are given n currencies $c_1, c_2, \ldots, c_n$ and an $n \times n$ table R of exchange rates, such that 1 unit of currency c_i buys $R[i, j]$ units of currency c_j.

a. Give an efficient algorithm to determine whether there exists a sequence of currencies $\langle c_{i_1}, c_{i_2}, \ldots, c_{i_k} \rangle$ such that

$$R[i_1, i_2] \cdot R[i_2, i_3] \cdots R[i_{k-1}, i_k] \cdot R[i_k, i_1] > 1 \ .$$

Analyze the running time of your algorithm.

b. Give an efficient algorithm to print out such a sequence if one exists. Analyze the running time of your algorithm.

22-4 Gabow's scaling algorithm for single-source shortest paths

A ***scaling*** algorithm solves a problem by initially considering only the highest-order bit of each relevant input value, such as an edge weight, assuming that these values are nonnegative integers. The algorithm then refines the initial solution by looking at the two highest-order bits. It progressively looks at more and more high-order bits, refining the solution each time, until it has examined all bits and computed the correct solution.

This problem examines an algorithm for computing the shortest paths from a single source by scaling edge weights. The input is a directed graph $G = (V, E)$ with nonnegative integer edge weights w. Let $W = \max\{w(u, v) : (u, v) \in E\}$ be the maximum weight of any edge. In this problem, you will develop an algorithm that runs in $O(E \lg W)$ time. Assume that all vertices are reachable from the source.

The scaling algorithm uncovers the bits in the binary representation of the edge weights one at a time, from the most significant bit to the least significant bit. Specifically, let $k = \lceil \lg(W + 1) \rceil$ be the number of bits in the binary representation of W, and for $i = 1, 2, \ldots, k$, let $w_i(u, v) = \lfloor w(u, v)/2^{k-i} \rfloor$. That is, $w_i(u, v)$ is the "scaled-down" version of $w(u, v)$ given by the i most significant bits of $w(u, v)$. (Thus, $w_k(u, v) = w(u, v)$ for all $(u, v) \in E$.) For example, if $k = 5$ and $w(u, v) = 25$, which has the binary representation $\langle 11001 \rangle$, then $w_3(u, v) = \langle 110 \rangle = 6$. Also with $k = 5$, if $w(u, v) = \langle 00100 \rangle = 4$, then $w_4(u, v) = \langle 0010 \rangle = 2$. Define $\delta_i(u, v)$ as the shortest-path weight from vertex u

to vertex v using weight function w_i, so that $\delta_k(u, v) = \delta(u, v)$ for all $u, v \in V$. For a given source vertex s, the scaling algorithm first computes the shortest-path weights $\delta_1(s, v)$ for all $v \in V$, then computes $\delta_2(s, v)$ for all $v \in V$, and so on, until it computes $\delta_k(s, v)$ for all $v \in V$. Assume throughout that $|E| \geq |V| - 1$. You will show how to compute δ_i from δ_{i-1} in $O(E)$ time, so that the entire algorithm takes $O(kE) = O(E \lg W)$ time.

a. Suppose that for all vertices $v \in V$, we have $\delta(s, v) \leq |E|$. Show how to compute $\delta(s, v)$ for all $v \in V$ in $O(E)$ time.

b. Show how to compute $\delta_1(s, v)$ for all $v \in V$ in $O(E)$ time.

Now focus on computing δ_i from δ_{i-1}.

c. Prove that for $i = 2, 3, \ldots, k$, either $w_i(u, v) = 2w_{i-1}(u, v)$ or $w_i(u, v) = 2w_{i-1}(u, v) + 1$. Then prove that

$$2\delta_{i-1}(s, v) \leq \delta_i(s, v) \leq 2\delta_{i-1}(s, v) + |V| - 1$$

for all $v \in V$.

d. Define, for $i = 2, 3, \ldots, k$ and all $(u, v) \in E$,

$$\widehat{w}_i(u, v) = w_i(u, v) + 2\delta_{i-1}(s, u) - 2\delta_{i-1}(s, v) .$$

Prove that for $i = 2, 3, \ldots, k$ and all $u, v \in V$, the "reweighted" value $\widehat{w}_i(u, v)$ of edge (u, v) is a nonnegative integer.

e. Now define $\widehat{\delta}_i(s, v)$ as the shortest-path weight from s to v using the weight function $\widehat{w}_i$. Prove that for $i = 2, 3, \ldots, k$ and all $v \in V$,

$$\delta_i(s, v) = \widehat{\delta}_i(s, v) + 2\delta_{i-1}(s, v)$$

and that $\widehat{\delta}_i(s, v) \leq |E|$.

f. Show how to compute $\delta_i(s, v)$ from $\delta_{i-1}(s, v)$ for all $v \in V$ in $O(E)$ time. Conclude that you can compute $\delta(s, v)$ for all $v \in V$ in $O(E \lg W)$ time.

22-5 Karp's minimum mean-weight cycle algorithm

Let $G = (V, E)$ be a directed graph with weight function $w : E \to \mathbb{R}$, and let $n = |V|$. We define the ***mean weight*** of a cycle $c = \langle e_1, e_2, \ldots, e_k \rangle$ of edges in E to be

$$\mu(c) = \frac{1}{k} \sum_{i=1}^{k} w(e_i) \, .$$

Let $\mu^* = \min \{\mu(c) : c \text{ is a directed cycle in } G\}$. We call a cycle c for which $\mu(c) = \mu^*$ a ***minimum mean-weight cycle***. This problem investigates an efficient algorithm for computing μ^*.

Assume without loss of generality that every vertex $v \in V$ is reachable from a source vertex $s \in V$. Let $\delta(s, v)$ be the weight of a shortest path from s to v, and let $\delta_k(s, v)$ be the weight of a shortest path from s to v consisting of *exactly* k edges. If there is no path from s to v with exactly k edges, then $\delta_k(s, v) = \infty$.

a. Show that if $\mu^* = 0$, then G contains no negative-weight cycles and $\delta(s, v) = \min \{\delta_k(s, v) : 0 \le k \le n - 1\}$ for all vertices $v \in V$.

b. Show that if $\mu^* = 0$, then

$$\max \left\{ \frac{\delta_n(s, v) - \delta_k(s, v)}{n - k} : 0 \le k \le n - 1 \right\} \ge 0$$

for all vertices $v \in V$. (*Hint:* Use both properties from part (a).)

c. Let c be a 0-weight cycle, and let u and v be any two vertices on c. Suppose that $\mu^* = 0$ and that the weight of the simple path from u to v along the cycle is x. Prove that $\delta(s, v) = \delta(s, u) + x$. (*Hint:* The weight of the simple path from v to u along the cycle is $-x$.)

d. Show that if $\mu^* = 0$, then on each minimum mean-weight cycle there exists a vertex v such that

$$\max \left\{ \frac{\delta_n(s, v) - \delta_k(s, v)}{n - k} : 0 \le k \le n - 1 \right\} = 0 \, .$$

(*Hint:* Show how to extend a shortest path to any vertex on a minimum mean-weight cycle along the cycle to make a shortest path to the next vertex on the cycle.)

e. Show that if $\mu^* = 0$, then the minimum value of

$$\max \left\{ \frac{\delta_n(s, v) - \delta_k(s, v)}{n - k} : 0 \le k \le n - 1 \right\} \, ,$$

taken over all vertices $v \in V$, equals 0.

f. Show that if you add a constant t to the weight of each edge of G, then μ^* increases by t. Use this fact to show that μ^* equals the minimum value of

$$\max\left\{\frac{\delta_n(s,v)-\delta_k(s,v)}{n-k} : 0 \le k \le n-1\right\},$$

taken over all vertices $v \in V$.

g. Give an $O(VE)$-time algorithm to compute μ^*.

22-6 Bitonic shortest paths

A sequence is ***bitonic*** if it monotonically increases and then monotonically decreases, or if by a circular shift it monotonically increases and then monotonically decreases. For example the sequences $\langle 1, 4, 6, 8, 3, -2\rangle$, $\langle 9, 2, -4, -10, -5\rangle$, and $\langle 1, 2, 3, 4\rangle$ are bitonic, but $\langle 1, 3, 12, 4, 2, 10\rangle$ is not bitonic. (See Problem 14-3 on page 407 for the bitonic euclidean traveling-salesperson problem.)

Suppose that you are given a directed graph $G = (V, E)$ with weight function $w : E \to \mathbb{R}$, where all edge weights are unique, and you wish to find single-source shortest paths from a source vertex s. You are given one additional piece of information: for each vertex $v \in V$, the weights of the edges along any shortest path from s to v form a bitonic sequence.

Give the most efficient algorithm you can to solve this problem, and analyze its running time.

Chapter notes

The shortest-path problem has a long history that is nicely desribed in an article by Schrijver [400]. He credits the general idea of repeatedly executing edge relaxations to Ford [148]. Dijkstra's algorithm [116] appeared in 1959, but it contained no mention of a priority queue. The Bellman-Ford algorithm is based on separate algorithms by Bellman [45] and Ford [149]. The same algorithm is also attributed to Moore [334]. Bellman describes the relation of shortest paths to difference constraints. Lawler [276] describes the linear-time algorithm for shortest paths in a dag, which he considers part of the folklore.

When edge weights are relatively small nonnegative integers, more efficient algorithms result from using min-priority queues that require integer keys and rely on the sequence of values returned by the EXTRACT-MIN calls in Dijkstra's algorithm monotonically increasing over time. Ahuja, Mehlhorn, Orlin, and Tarjan [8] give an algorithm that runs in $O(E + V\sqrt{\lg W})$ time on graphs with nonnegative edge weights, where W is the largest weight of any edge in the

graph. The best bounds are by Thorup [436], who gives an algorithm that runs in $O(E \lg\lg V)$ time, and by Raman [375], who gives an algorithm that runs in $O\left(E + V \min\left\{(\lg V)^{1/3+\epsilon}, (\lg W)^{1/4+\epsilon}\right\}\right)$ time. These two algorithms use an amount of space that depends on the word size of the underlying machine. Although the amount of space used can be unbounded in the size of the input, it can be reduced to be linear in the size of the input using randomized hashing.

For undirected graphs with integer weights, Thorup [435] gives an algorithm that runs in $O(V + E)$ time for single-source shortest paths. In contrast to the algorithms mentioned in the previous paragraph, the sequence of values returned by EXTRACT-MIN calls does not monotonically increase over time, and so this algorithm is not an implementation of Dijkstra's algorithm. Pettie and Ramachandran [357] remove the restriction of integer weights on undirected graphs. Their algorithm entails a preprocessing phase, followed by queries for specific source vertices. Preprocessing takes $O(MST(V, E) + \min\{V \lg V, V \lg\lg r\})$ time, where $MST(V, E)$ is the time to compute a minimum spanning tree and r is the ratio of the maximum edge weight to the minimum edge weight. After preprocessing, each query takes $O(E \lg \widehat{\alpha}(E, V))$ time, where $\widehat{\alpha}(E, V)$ is the inverse of Ackermann's function. (See the chapter notes for Chapter 19 for a brief discussion of Ackermann's function and its inverse.)

For graphs with negative edge weights, an algorithm due to Gabow and Tarjan [167] runs in $O(\sqrt{V}E \lg(VW))$ time, and one by Goldberg [186] runs in $O(\sqrt{V}E \lg W)$ time, where $W = \max\{|w(u, v)| : (u, v) \in E\}$. There has also been some progress based on methods that use continuous optimization and electrical flows. Cohen et al. [98] give such an algorithm, which is randomized and runs in $\widetilde{O}(E^{10/7} \lg W)$ expected time (see Problem 3-6 on page 73 for the defintion of $\widetilde{O}$-notation). There is also a pseudopolyomial-time algorithm based on fast matrix multiplication. Sankowski [394] and Yuster and Zwick [465] designed an algorithm for shortest paths that runs in $\widetilde{O}(WV^{\omega})$ time, where two $n \times n$ matrices can be multiplied in $O(n^{\omega})$ time, giving a faster algorithm than the previously mentioned algorithms for small values of W on dense graphs.

Cherkassky, Goldberg, and Radzik [89] conducted extensive experiments comparing various shortest-path algorithms. Shortest-path algorithms are widely used in real-time navigation and route-planning applications. Typically based on Dijkstra's algorithm, these algorithms use many clever ideas to be able to compute shortest paths on networks with many millions of vertices and edges in fractions of a second. Bast et al. [36] survey many of these developments.

23 All-Pairs Shortest Paths

In this chapter, we turn to the problem of finding shortest paths between all pairs of vertices in a graph. A classic application of this problem occurs in computing a table of distances between all pairs of cities for a road atlas. Classic perhaps, but not a true application of finding shortest paths between *all* pairs of vertices. After all, a road map modeled as a graph has one vertex for *every* road intersection and one edge wherever a road connects intersections. A table of intercity distances in an atlas might include distances for 100 cities, but the United States has approximately 300,000 signal-controlled intersections[1] and many more uncontrolled intersections.

A legitimate application of all-pairs shortest paths is to determine the ***diameter*** of a network: the longest of all shortest paths. If a directed graph models a communication network, with the weight of an edge indicating the time required for a message to traverse a communication link, then the diameter gives the longest possible transit time for a message in the network.

As in Chapter 22, the input is a weighted, directed graph $G = (V, E)$ with a weight function $w : E \to \mathbb{R}$ that maps edges to real-valued weights. Now the goal is to find, for every pair of vertices $u, v \in V$, a shortest (least-weight) path from u to v, where the weight of a path is the sum of the weights of its constituent edges. For the all-pairs problem, the output typically takes a tabular form in which the entry in u's row and v's column is the weight of a shortest path from u to v.

You can solve an all-pairs shortest-paths problem by running a single-source shortest-paths algorithm $|V|$ times, once with each vertex as the source. If all edge weights are nonnegative, you can use Dijkstra's algorithm. If you implement the min-priority queue with a linear array, the running time is $O(V^3 + VE)$ which is $O(V^3)$. The binary min-heap implementation of the min-priority queue

---

[1] According to a report cited by U.S. Department of Transportation Federal Highway Administration, "a reasonable 'rule of thumb' is one signalized intersection per 1,000 population."

yields a running time of $O(V(V + E) \lg V)$. If $|E| = \Omega(V)$, the running time becomes $O(VE \lg V)$, which is faster than $O(V^3)$ if the graph is sparse. Alternatively, you can implement the min-priority queue with a Fibonacci heap, yielding a running time of $O(V^2 \lg V + VE)$.

If the graph contains negative-weight edges, Dijkstra's algorithm doesn't work, but you can run the slower Bellman-Ford algorithm once from each vertex. The resulting running time is $O(V^2 E)$, which on a dense graph is $O(V^4)$. This chapter shows how to guarantee a much better asymptotic running time. It also investigates the relation of the all-pairs shortest-paths problem to matrix multiplication.

Unlike the single-source algorithms, which assume an adjacency-list representation of the graph, most of the algorithms in this chapter represent the graph by an adjacency matrix. (Johnson's algorithm for sparse graphs, in Section 23.3, uses adjacency lists.) For convenience, we assume that the vertices are numbered $1, 2, \ldots, |V|$, so that the input is an $n \times n$ matrix $W = (w_{ij})$ representing the edge weights of an n-vertex directed graph $G = (V, E)$, where

$$w_{ij} = \begin{cases} 0 & \text{if } i = j \ , \\ \text{the weight of directed edge } (i, j) & \text{if } i \neq j \text{ and } (i, j) \in E \ , \\ \infty & \text{if } i \neq j \text{ and } (i, j) \notin E \ . \end{cases} \tag{23.1}$$

The graph may contain negative-weight edges, but we assume for the time being that the input graph contains no negative-weight cycles.

The tabular output of each of the all-pairs shortest-paths algorithms presented in this chapter is an $n \times n$ matrix. The (i, j) entry of the output matrix contains $\delta(i, j)$, the shortest-path weight from vertex i to vertex j, as in Chapter 22.

A full solution to the all-pairs shortest-paths problem includes not only the shortest-path weights but also a ***predecessor matrix*** $\Pi = (\pi_{ij})$, where π_{ij} is NIL if either $i = j$ or there is no path from i to j, and otherwise π_{ij} is the predecessor of j on some shortest path from i. Just as the predecessor subgraph G_π from Chapter 22 is a shortest-paths tree for a given source vertex, the subgraph induced by the ith row of the Π matrix should be a shortest-paths tree with root i. For each vertex $i \in V$, the ***predecessor subgraph*** of G for i is $G_{\pi,i} = (V_{\pi,i}, E_{\pi,i})$, where

$$V_{\pi,i} = \{j \in V : \pi_{ij} \neq \text{NIL}\} \cup \{i\} \ ,$$
$$E_{\pi,i} = \{(\pi_{ij}, j) : j \in V_{\pi,i} - \{i\}\} \ .$$

If $G_{\pi,i}$ is a shortest-paths tree, then PRINT-ALL-PAIRS-SHORTEST-PATH on the following page, which is a modified version of the PRINT-PATH procedure from Chapter 20, prints a shortest path from vertex i to vertex j.

In order to highlight the essential features of the all-pairs algorithms in this chapter, we won't cover how to compute predecessor matrices and their properties as extensively as we dealt with predecessor subgraphs in Chapter 22. Some of the exercises cover the basics.

```
PRINT-ALL-PAIRS-SHORTEST-PATH(Π, i, j)
1  if i == j
2       print i
3  elseif π_ij == NIL
4       print "no path from" i "to" j "exists"
5  else PRINT-ALL-PAIRS-SHORTEST-PATH(Π, i, π_ij)
6       print j
```

Chapter outline

Section 23.1 presents a dynamic-programming algorithm based on matrix multiplication to solve the all-pairs shortest-paths problem. The technique of "repeated squaring" yields a running time of $\Theta(V^3 \lg V)$. Section 23.2 gives another dynamic-programming algorithm, the Floyd-Warshall algorithm, which runs in $\Theta(V^3)$ time. Section 23.2 also covers the problem of finding the transitive closure of a directed graph, which is related to the all-pairs shortest-paths problem. Finally, Section 23.3 presents Johnson's algorithm, which solves the all-pairs shortest-paths problem in $O(V^2 \lg V + VE)$ time and is a good choice for large, sparse graphs.

Before proceeding, we need to establish some conventions for adjacency-matrix representations. First, we generally assume that the input graph $G = (V, E)$ has n vertices, so that $n = |V|$. Second, we use the convention of denoting matrices by uppercase letters, such as W, L, or D, and their individual elements by subscripted lowercase letters, such as w_{ij}, l_{ij}, or d_{ij}. Finally, some matrices have parenthesized superscripts, as in $L^{(r)} = \left(l_{ij}^{(r)}\right)$ or $D^{(r)} = \left(d_{ij}^{(r)}\right)$, to indicate iterates.

23.1 Shortest paths and matrix multiplication

This section presents a dynamic-programming algorithm for the all-pairs shortest-paths problem on a directed graph $G = (V, E)$. Each major loop of the dynamic program invokes an operation similar to matrix multiplication, so that the algorithm looks like repeated matrix multiplication. We'll start by developing a $\Theta(V^4)$-time algorithm for the all-pairs shortest-paths problem, and then we'll improve its running time to $\Theta(V^3 \lg V)$.

Before proceeding, let's briefly recap the steps given in Chapter 14 for developing a dynamic-programming algorithm:

1. Characterize the structure of an optimal solution.
2. Recursively define the value of an optimal solution.
3. Compute the value of an optimal solution in a bottom-up fashion.